# SIMATS ENGINEERING

ASSESSMENT TOOL 1

---

# ASSESSMENT

Course Name: INTERNET PROGRAMMING

Course Code: CSA43

Course Outcome Covered: CO-1

Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

| | |
|---|---|
| **Student Name** | Naveen.S |
| **Register Number** | 192324248 |
| **Assessment Tool** | Application-Based Questions |
| **Weightage** | 40% |
| **Total Marks** | 30 |

Submission note: This completed document preserves the original assessment order and responds to every listed sub-question.

# Assessment Information

The source assessment contains three application-based scenarios covering HTML5 semantics, standards compliance, HTTP request/response behaviour, cross-browser compatibility, form submission, GET semantics, and secure transmission of form data.

## Course Outcome (CO-1)

Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

---

# Question 1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

### 1.1 Identify appropriate HTML5 semantic elements missing in the structure.

Answer: The form is functional, but it lacks structural and semantic elements that communicate the purpose of each part of the page to browsers, assistive technologies, and developers. Appropriate elements include <header> for the portal heading/branding, <main> for the primary registration content, <section> for the registration section, and <footer> for supporting information. Within the form, <label> elements should explicitly identify the name and email fields. A <fieldset> and <legend> can also group and describe the registration controls.

These elements improve semantic meaning and accessibility without changing the underlying HTTP mechanism.

### 1.2 Modify the structure to make it standards-compliant.

Answer: A standards-compliant version should include the HTML5 document structure, language declaration, character encoding, semantic page regions, explicit labels, suitable input attributes, and a submit control.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>College Symposium Registration</h1>
  </header>

  <main>
    <section aria-labelledby="registration-heading">
      <h2 id="registration-heading">Register for the Symposium</h2>

      <form action="/register" method="post">
        <fieldset>
          <legend>Student Details</legend>
```

```
        <label for="name">Full Name:</label>
        <input type="text" id="name" name="name"
               autocomplete="name" required>

        <label for="email">Email Address:</label>
        <input type="email" id="email" name="email"
               autocomplete="email" required>

        <button type="submit">Submit Registration</button>
      </fieldset>
    </form>
  </section>
 </main>

 <footer>
   <p>&copy; College Symposium</p>
 </footer>
</body>
</html>
```

### 1.3 Interpret the HTTP request generated during form submission.

Answer: Because the form specifies method="POST" and action="/register", clicking the submit button causes the browser to create an HTTP POST request for the /register resource on the current server. The submitted controls are encoded in the request body rather than appended to the URL query string.

```
POST /register HTTP/1.1
Host: example.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: ...

name=Naveen.S&email=naveen%40example.com
```

The exact request also contains headers such as Host, User-Agent, Accept, and connection-related fields. The server receives the request, reads the body, validates the supplied values, and performs the registration operation. HTTPS should be used in a real application so the request is protected in transit.

### 1.4 Analyze the HTTP response cycle from server to browser.

Answer: The response cycle is a request–response sequence:

| Stage | Operation |
|---|---|
| 1. Submit | The user activates the submit control and the browser validates required fields. |
| 2. Request | The browser sends a POST request to /register with form data in the request body. |
| 3. Server processing | The server receives, validates, sanitizes, and processes the registration data. |
| 4. Response | The server returns an HTTP status code, headers, and usually an HTML/JSON response body. |
| 5. Browser handling | The browser interprets the response, renders the returned content, or follows a redirect if one is s |
| 6. Completion | The user sees a success page, validation message, or other appropriate result. |

For a successful creation request, a server may respond with 201 Created; for a normal successful processing response, 200 OK may be used. If the server redirects the browser to a confirmation page, a suitable 3xx response such as 303 See Other may be used. Error conditions can produce 4xx or 5xx responses.

### 1.5 Suggest improvements for usability and accessibility.

Answer: The portal can be improved by:

• Providing visible, correctly associated labels for every form control.
• Marking mandatory fields with required and giving clear instructions before submission.
• Using suitable HTML5 input types such as email and useful autocomplete values.
• Giving meaningful validation messages and identifying the field that needs correction.
• Maintaining a logical keyboard tab order and ensuring all controls are keyboard accessible.
• Using semantic landmarks such as header, main, section, and footer to help assistive technologies navigate the page.
• Providing sufficient text contrast, visible focus indicators, and responsive layout for mobile devices.
• Preventing accidental loss of entered information and displaying a clear confirmation after successful registration.
• Using HTTPS and server-side validation in addition to client-side validation.

# Question 2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
  <title>Company</title>
 </head>
 <body>
  <div>
   <h1>Welcome
   <p>About us
  </div>
 </body>
</html>
```

### 2.1 Identify improper nesting and missing closing tags.

Answer: The <h1> element is opened but not explicitly closed before the  element begins, and the paragraph is also not explicitly closed. The intended structure should close </h1> and  in the correct order before closing the containing . In addition, the document lacks the HTML5 doctype and a language declaration.

Correct nesting follows a parent–child hierarchy. An element opened inside another element should be closed before its parent is closed. Explicit closing tags make the DOM predictable and improve conformance.

### 2.2 Analyze how different browsers may interpret this structure.

Answer: HTML parsers attempt error recovery when markup is malformed. Browsers may automatically infer missing end tags and construct a DOM tree based on their parsing rules. Although modern browsers often produce a usable page, the recovered DOM is not guaranteed to match the developer's intended structure in every edge case.

Differences in parser recovery can influence element relationships, default styling, CSS selectors, accessibility-tree interpretation, and script behaviour. Consequently, the same malformed source can produce inconsistent visual or behavioural results across browsers and versions. Valid markup reduces this uncertainty.

### 2.3 Identify missing semantic elements (e.g., header, section).

Answer: Suitable semantic elements include <header> for the company/site heading, <main> for the principal page content, and <section> for the “About Us” content when it represents a distinct thematic section. A <footer> can be used for footer information. The generic  should be reserved for grouping when no more meaningful semantic element applies.

### 2.4 Provide a corrected W3C-compliant HTML structure.

Answer: The following version uses valid HTML5 structure and explicit, correctly nested elements.

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Company</title>
</head>
<body>
 <header>
  <h1>Welcome</h1>
 </header>
```

```
 <main>
  <section aria-labelledby="about-heading">
   <h2 id="about-heading">About Us</h2>
   <p>About us</p>
  </section>
 </main>

 <footer>
  <p>&copy; Company</p>
 </footer>
</body>
</html>
```

This structure has a doctype, language attribute, character encoding, responsive viewport declaration, valid nesting, semantic landmarks, and properly closed elements.

## 2.5 Suggest techniques to ensure cross-browser compatibility.

Answer: Recommended techniques include:

• Follow current HTML5 and CSS standards and validate markup.
• Use a standards mode doctype: <!DOCTYPE html>.
• Test on major browser engines and representative desktop/mobile versions.
• Use feature detection rather than assuming that a browser supports a feature.
• Apply a consistent CSS reset or carefully defined baseline styles when appropriate.
• Avoid obsolete, vendor-specific, or non-standard markup unless a justified compatibility strategy exists.
• Use progressive enhancement so essential content and functionality remain available when advanced features are unsupported.
• Use automated validation, linting, and browser-based testing during development.
• Check responsive behaviour at different viewport sizes and input methods.
• Keep CSS and JavaScript standards-compliant and test accessibility as part of compatibility testing.

# Question 3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

### 3.1 Identify issues in the form configuration.

Answer: The main problem is the button type. <button type="button"> is a general-purpose button and does not submit the form. It must be type="submit" (or the type can be omitted because submit is the default for a button associated with a form).

There are also security and design concerns. The form uses GET while collecting a password, which is inappropriate because GET form values are normally encoded into the URL query string. The fields also lack explicit labels and useful validation attributes such as required.

### 3.2 Analyze why the HTTP request is not triggered.

Answer: The browser submits a form when a submit button is activated, when an appropriate form-submission mechanism is invoked, or when submission is triggered programmatically. In the given markup, type="button" creates a non-submitting button. Clicking it therefore does not initiate the form's normal submission algorithm, so no request is generated merely by clicking “Login”.

Changing the control to type="submit" causes the browser to validate the form and then construct the HTTP request using the form's action and method, subject to any validation or JavaScript behaviour.

### 3.3 Interpret the role of GET method in this scenario.

Answer: GET is intended primarily for retrieving a resource or performing a safe/read-oriented operation. For an HTML form, successful controls are generally encoded into the URL query string, for example:

```
GET /submit?username=student&password=example HTTP/1.1
Host: example.edu
```

This is unsuitable for password submission because URLs may be stored in browser history, server/proxy logs, analytics systems, bookmarks, or other intermediary records. Even when HTTPS encrypts the connection, the sensitive value should not be placed in the URL in the first place.

Therefore, GET is appropriate for non-sensitive query parameters such as search terms or filters, while a login form should normally use POST over HTTPS.

### 3.4 Propose a corrected implementation.

Answer: A corrected implementation uses POST, a submit button, explicit labels, required fields, autocomplete hints, and an appropriate action endpoint.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
  <main>
```

```
    <section aria-labelledby="login-heading">
      <h1 id="login-heading">Login</h1>

      <form action="/submit" method="post">
        <div>
          <label for="username">Username:</label>
          <input type="text" id="username" name="username"
                 autocomplete="username" required>
        </div>

        <div>
          <label for="password">Password:</label>
          <input type="password" id="password" name="password"
                 autocomplete="current-password" required>
        </div>

        <button type="submit">Login</button>
      </form>
    </section>
  </main>
</body>
</html>
```

The server should authenticate the submitted credentials securely and should return an appropriate status/result. The endpoint must be served through HTTPS in production.

### 3.5 Suggest improvements for secure data transmission.

Answer: Secure transmission should combine transport security, secure server processing, and appropriate credential handling:

• Use HTTPS with a valid TLS configuration for the entire login flow.
• Submit credentials with POST, not GET, so they are not placed in the URL query string.
• Validate input on the server; client-side validation alone is not a security boundary.
• Store passwords only as strong password hashes using a modern password-hashing algorithm and appropriate salt/work factor; never store plaintext passwords.
• Use secure, appropriately configured session cookies after authentication, including protections such as Secure and HttpOnly where applicable, and suitable SameSite settings.
• Protect authentication endpoints against common attacks such as credential stuffing and excessive automated attempts through rate limiting and monitoring.
• Do not expose passwords or authentication tokens in URLs, logs, error messages, or client-side source code.
• Return clear but non-revealing authentication errors so attackers cannot easily enumerate accounts.
• Keep the server, framework, dependencies, and TLS configuration maintained and patched.